

PROJECT REPORT

Shobhit Institute of Engineering & Technology, Meerut

(NAAC 'A' Grade Accredited Deemed-to-be University)

NH-58, Modipuram, Meerut - 250 110

MailForge – SaaS-Based Email Campaign & Analytics Platform

SESSION: 2025-2026

[T01BCCA0496]

Submitted by:

Name: Gupta Karan

Cours: BCA 2nd Year

Roll No: - MRT24UGBCA114

Enroll No: - 20240110548

Name: Mayank Raj

Cours: BCA 2nd Year

Roll No: - MRT24UGBCA127

Enroll No: - 20240110640

Submitted to:

Ms. Priyanka Saini (Assistant Professor)



Department of Computer Science and Engineering

CERTIFICATE

This is to certify that **Gupta Karan** and **Mayank Raj**, students of **BCA 2nd Year, Section B**, have successfully completed the Minor Project titled “**MailForge – SaaS-Based Email Campaign & Analytics Platform**” under the guidance of **Ms. Priyanka Saini**, Department of Computer Science and Engineering, during the academic session **2025–2026**.

Project Title: MailForge – SaaS-Based Email Campaign & Analytics Platform

Date of Completion: March 9, 2026

Name of the External Examiner

Name of the Internal Examiner

Signature

Signature

ACKNOWLEDGMENT

We express our heartfelt gratitude to everyone who contributed to the successful completion of our project.

We sincerely thank **Ms. Priyanka Saini, Assistant Professor, Department of Computer Science & Engineering**, for her continuous support, motivation, and valuable guidance throughout the development of this project. Her insights and suggestions played a crucial role in shaping the project and improving the overall quality of our work.

We are also thankful to our Head of Department, respected faculty members, seniors, and classmates who provided support and suggestions at various stages of this project.

INDEX

<u>SL.NO</u>	<u>DESCRIPTION</u>	<u>PAGE NO</u>
1.	Introduction	i
2.	Problem Statement	ii
3.	Feasibility Study	iii
4.	Project Architecture	iv
5.	Technology Used in the Project	v
6.	Work Flow	vi
7.	Flow Diagram	vii-viii
8.	Project Screenshots	ix-x
9.	Conclusion	xi
10.	References	xii

INTRODUCTION

MailForge is a high-performance, SaaS-style bulk email application that handles the complete lifecycle of an email campaign — from composing emails to tracking delivery results. It is built using the Django web framework and is designed for organizations, colleges, or individuals who need to send large volumes of personalized emails efficiently.

In today's digital world, sending personalized emails at scale — such as event invitations, results, or certificates — is a common need. Traditional manual methods are slow and error-prone. MailForge solves this by automating the entire process: upload a contact list, compose your email with personalized variables, attach auto-generated certificates, and send — all from one platform.

The application features a Single-Page Application (SPA) architecture powered by Hotwire Turbo, meaning users can navigate between different sections — Dashboard, Compose, Analytics, and Settings — without any page reloads, giving a smooth and modern experience.

Key Features of MailForge:

1. Bulk Email Sending — Send thousands of emails simultaneously using multi-threaded processing.
2. Rich Text Composer — Write formatted HTML emails using the built-in Quill.js editor.
3. Dynamic Personalization — Use tags like @Name, @College, @Event to auto-fill each recipient's details.
4. Certificate Generation — Automatically create and attach personalized PDF certificates to each email.
5. Excel Contact Import — Upload recipient lists via .xlsx files in one click.
6. Campaign Analytics — Track delivery rates and success/failure ratios through live charts.
7. Smart Attachments — Auto-compress multiple files into a single ZIP before sending.
8. Dynamic SMTP Settings — Configure email server details directly from the UI without touching code.
9. Draft Auto-Save — Email drafts are saved automatically in the browser to prevent data loss.
10. Live Certificate Preview — Preview the certificate with a dummy name before sending to all recipients.

PROBLEM STATEMENT

Title: Automating Bulk Email Campaigns with Personalized Certificate Delivery

Problem:

Organizations frequently need to send large volumes of personalized emails — such as event certificates, participation confirmations, or announcements. Currently, this is done manually, which creates several problems:

11. Sending individual emails is extremely time-consuming for large contact lists.
12. Manually creating and attaching certificates for each recipient is repetitive and error-prone.
13. No centralized platform exists to manage contacts, compose emails, and track delivery in one place.
14. Standard email clients do not support bulk personalization (inserting each recipient's name automatically).
15. Tracking which emails were delivered, failed, or opened is not possible with manual methods.
16. Attaching multiple files increases email size and slows down sending.

Objective:

To develop a secure, high-performance, web-based email campaign platform that automates the complete process of bulk email sending, dynamic certificate generation, and delivery tracking through an intuitive interface.

Key Goals:

- : Automate personalized bulk email delivery at scale
- : Dynamically generate and attach PDF certificates per recipient
- : Provide real-time campaign analytics and delivery tracking
- : Allow SMTP configuration without touching server code
- : Deliver a fast, seamless Single-Page Application experience

Impact:

- : Reduces hours of manual email work to just a few minutes
- : Eliminates human error in certificate generation and distribution
- : Provides organizations with professional-grade campaign management tools
- : Saves significant time and effort for educational institutions and event organizers

FEASIBILITY STUDY

1. Technical Feasibility

The project is technically feasible as it is built entirely on well-established, open-source technologies. Django (Python) provides a robust backend with excellent support for file handling, email dispatch, and database management. The frontend uses Bootstrap 5 and standard HTML5/CSS3, supported by all modern browsers. Libraries such as Pillow for image processing and Quill.js for rich text editing are mature and well-documented. Hotwire Turbo enables the SPA experience without requiring a separate JavaScript framework.

2. Operational Feasibility

The user interface is clean and intuitive, resembling modern SaaS email tools. Users only need basic computer knowledge to operate the platform. The workflow — upload contacts, compose email, preview certificate, and send — is straightforward and requires no technical training. The admin settings panel allows SMTP configuration through simple form fields, making it accessible to non-technical administrators.

3. Economic Feasibility

The entire project is built using free and open-source technologies. Django, Python, PostgreSQL, Bootstrap 5, Quill.js, and Hotwire Turbo all carry zero licensing costs. The only potential cost is cloud hosting for deployment (e.g., a basic VPS server), which is minimal. This makes MailForge a highly cost-effective solution for organizations of any size.

PROJECT ARCHITECTURE

MailForge follows a clean, layered architecture where each component has a specific responsibility. Below is a breakdown of how the system is structured:

1. Client Interface (Frontend)

Users interact with the system through responsive web pages built with HTML5, CSS3 (Bootstrap 5), and JavaScript. The main pages include: Dashboard, Compose Email, Recipient Manager, Campaign Logs, and SMTP Settings.

2. SPA Navigation (Hotwire Turbo)

Hotwire Turbo intercepts all navigation links and form submissions, fetching only the changed part of the page from the server and updating the DOM. This eliminates full-page reloads. JavaScript components like Chart.js and Quill.js are re-initialized on each turbo:load event to prevent duplication.

3. Django Views & Business Logic

Django views handle all core application logic: processing email composition forms, dispatching bulk emails using ThreadPoolExecutor (4 concurrent workers), generating PDF certificates in memory, compressing attachments into ZIP files, and managing SMTP connection testing.

4. Database & Models (PostgreSQL)

PostgreSQL is the primary database. Key models include: Recipient (contact details with JSONField), EmailLog (delivery status per recipient), Campaign (campaign history), and SMTPConfig (dynamic server configuration). Bulk database writes use bulk_create() for maximum performance.

5. Email Dispatch Engine

The engine loads SMTP settings from the database, personalizes the email body by replacing tags (@Name, @College, @Event), generates a PDF certificate in memory using Pillow, compresses attachments with io.BytesIO (no disk I/O), and dispatches via 4 parallel threads simultaneously.

6. Analytics & Logging

Every email sent is logged in the EmailLog table with delivery status and timestamps. The Analytics page reads these logs and renders a Line Chart (emails over time) and a Donut Chart (success vs. failure) using Chart.js.

7. Admin Settings Panel

A database-driven settings page allows the administrator to update SMTP Host, Port, Sender Email, and App Password from the UI. A built-in AJAX connection tester verifies credentials before any campaign is launched.

TECHNOLOGY USED IN THE PROJECT

Frontend

- **HTML5:** Structure and layout of all web pages
- **CSS3 / Bootstrap 5:** Responsive styling with a modern dark-sidebar SaaS design
- **JavaScript:** Dynamic UI interactions, AJAX calls, and form validation
- **Quill.js:** Rich text editor for composing HTML emails
- **Chart.js:** Renders Line Charts and Donut Charts on the Analytics page
- **Hotwire Turbo:** Enables Single-Page Application behavior — seamless navigation without full page reloads

Backend

- **Python (Django):** Core application logic, URL routing, ORM, authentication, and email dispatch
- **Pillow (PIL):** Image processing library for generating personalized certificates
- **concurrent.futures:** Python's ThreadPoolExecutor for parallel, multi-threaded email sending
- **io.BytesIO:** In-memory file processing for certificates and ZIP compression — no disk I/O

Database

- **PostgreSQL:** Primary relational database with JSONField support for flexible recipient data

Tools & Platforms

- **GitHub:** Version control and source code management
- **VS Code:** Primary code editor
- **Google App Passwords:** Required for Gmail SMTP authentication (16-digit app password)
- **Python pip:** Package manager for all Python dependencies

Hardware Requirements

- **RAM:** Minimum 4 GB recommended
- **Processor:** Dual-core or higher
- **Storage:** Minimum 200 MB for application and database
- **Network:** Stable internet connection required for SMTP email dispatch

WORK FLOW

The following describes the end-to-end workflow of the MailForge system, from user login to campaign completion.

1. Authentication

- **User Action:** Enter username and password on the login page
- **System Process:** Django verifies credentials against the database
- **Outcome:** Access granted to the private dashboard, or error message shown

2. SMTP Configuration

- **User Action:** Go to Settings and enter SMTP Host, Port, Sender Email, and App Password
- **System Process:** Settings are saved to the database. User can Test Connection via AJAX before proceeding
- **Outcome:** System is ready to dispatch emails using the configured server

3. Recipient Management

- **User Action:** Upload an Excel file (.xlsx) with columns: Name, College, Year, Email, Mobile, Event
- **System Process:** Django reads and parses each row, saving contacts to the database in bulk
- **Outcome:** Contact list appears in a scrollable, searchable data grid

4. Email Composition

- **User Action:** Write subject and body using the Quill.js editor. Insert personalization tags like @Name, @College
- **System Process:** Draft is auto-saved to LocalStorage to prevent data loss on reload
- **Outcome:** A personalized email template is ready for the campaign

5. Certificate Setup (Optional)

- **User Action:** Upload a blank certificate image (PNG/JPG). Define X/Y coordinates, font size, and text color for name placement. Click Preview to see a live preview
- **System Process:** Django uses Pillow to render the certificate in memory and returns a Base64 preview via AJAX
- **Outcome:** Certificate template is verified before sending

6. Campaign Dispatch

- **User Action:** Click Send Campaign
- **System Process:** For each recipient: tags are replaced with real data → certificate PDF is generated → files are zipped → email is dispatched. This runs across 4 parallel threads, processing 20 emails per batch simultaneously
- **Outcome:** Emails sent rapidly. All results logged via bulk_create()

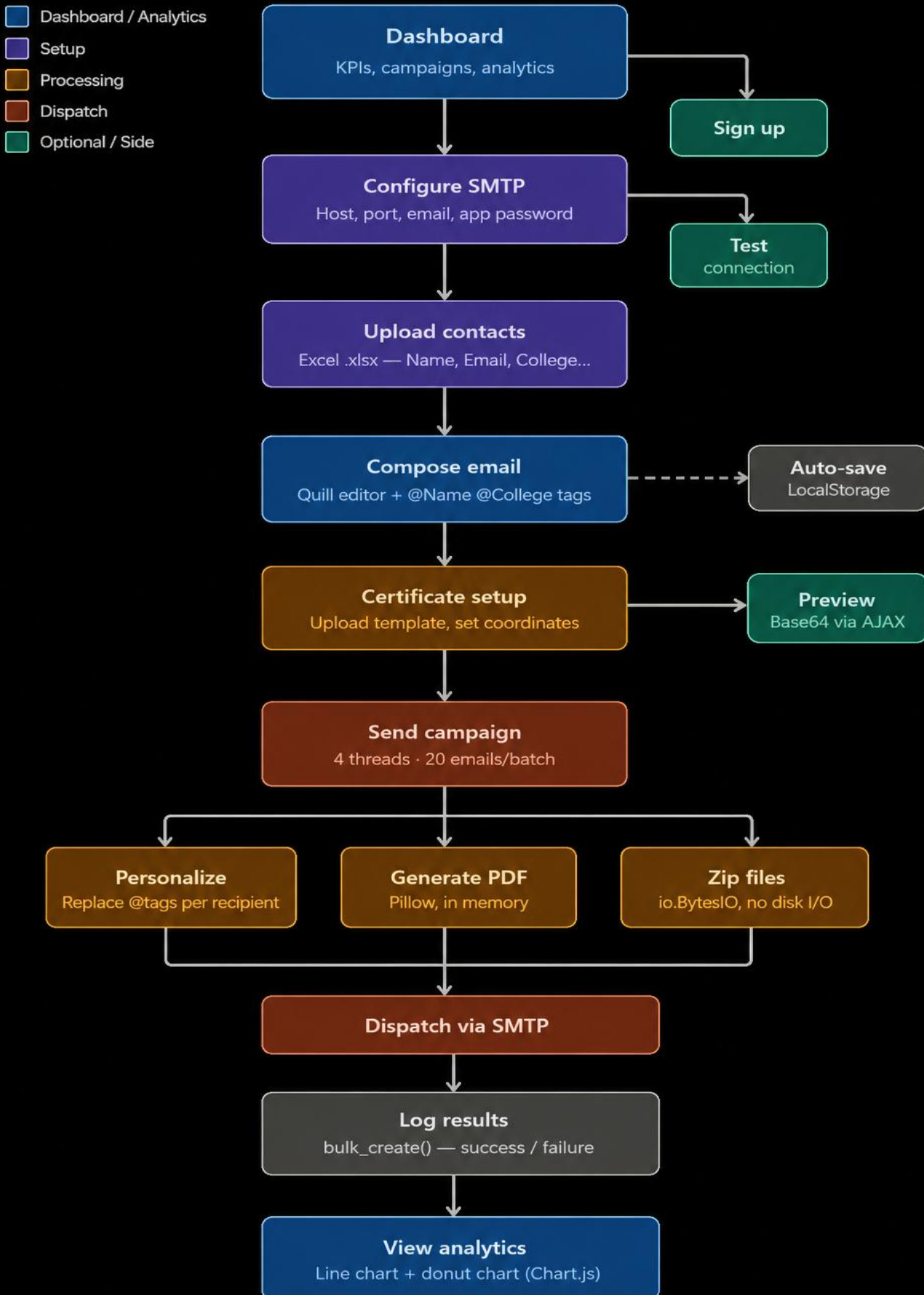
7. Analytics Review

- **User Action:** Open the Analytics or Sent Mails Log page
- **System Process:** Django queries the EmailLog table. Chart.js renders delivery trend and success/failure charts
- **Outcome:** User sees total delivered, failed emails, failure reasons, and delivery trends

8. Logout

- **Action:** User clicks Logout
- **Outcome:** Django destroys the session and redirects to the login page

FLOW DIAGRAM



The flow diagram below illustrates the complete system workflow of MailForge — from user authentication through to campaign completion and analytics reporting.

Detailed Workflow Explanation:

1. Authentication Phase

- **Start:** User opens the MailForge web application
- **Check:** Is the user logged in?
 - **No:** Redirect to Login/Signup Page. System authenticates against the database
 - **Yes:** Redirect to the Home Dashboard

2. Dashboard (Central Hub)

- **Initial Load:** Dashboard loads and displays KPIs: total contacts, campaigns run, emails delivered, and failures
- **System Action:** Fetches campaign data from the database and renders analytics charts
- **Display:** User sees a complete summary of all past campaign activity

3. Campaign Setup

- **User Action:** Uploads Excel contacts, configures SMTP settings, and composes the email
- **Optional:** Upload a certificate template and configure name placement coordinates
- **System Process:** Validates all inputs and confirms SMTP connectivity
- **Result:** System is ready to launch the campaign

4. Email Dispatch Phase

- **Trigger:** User clicks Send Campaign
- **Processing:** ThreadPoolExecutor distributes work across 4 concurrent threads, handling 20 emails per batch
- **Per Email:** Tags replaced → Certificate generated → Files zipped → Email dispatched → Result logged
- **Result:** Campaign completes. All logs saved via `bulk_create()`

5. Analytics Review

- **User Action:** Opens Analytics or Sent Mails Log page
- **System Action:** Queries EmailLog table and renders Line Chart and Donut Chart via Chart.js
- **Result:** User views detailed success/failure data and delivery trends

6. Logout

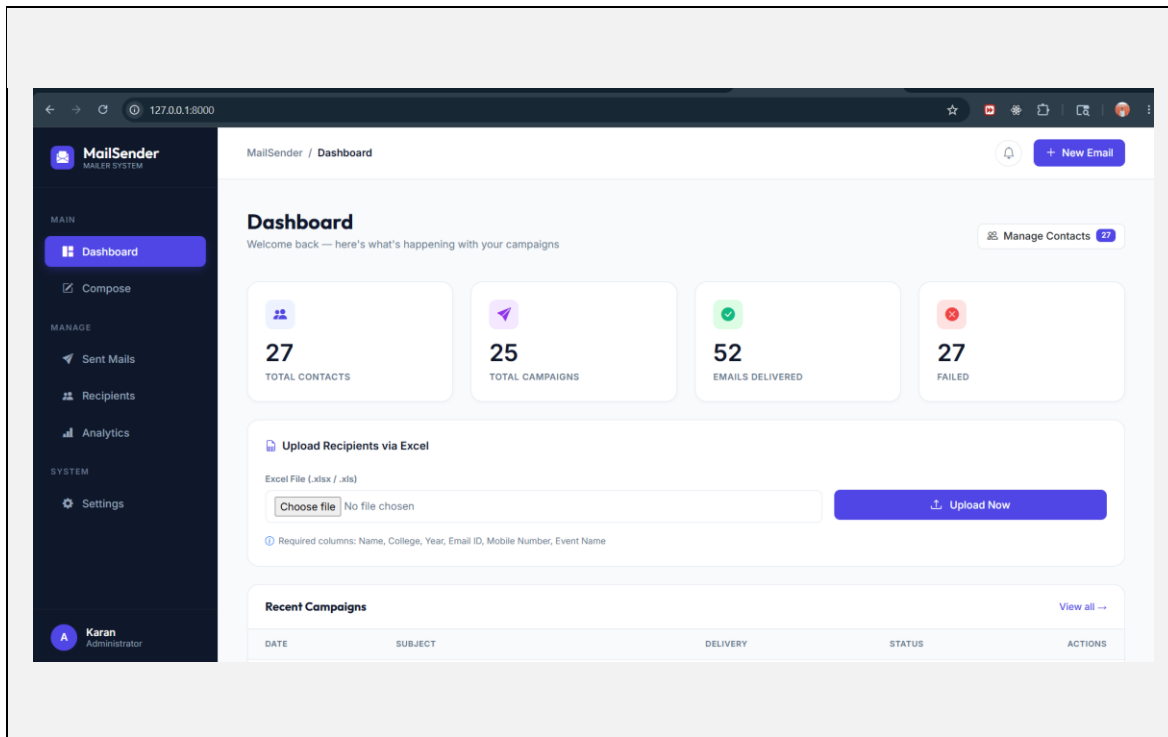
- **Action:** User clicks Logout
- **Outcome:** Session is destroyed and user is redirected to the login page

PROJECT SCREENSHOTS

The following screenshots demonstrate the key modules and user interface of the MailForge application.

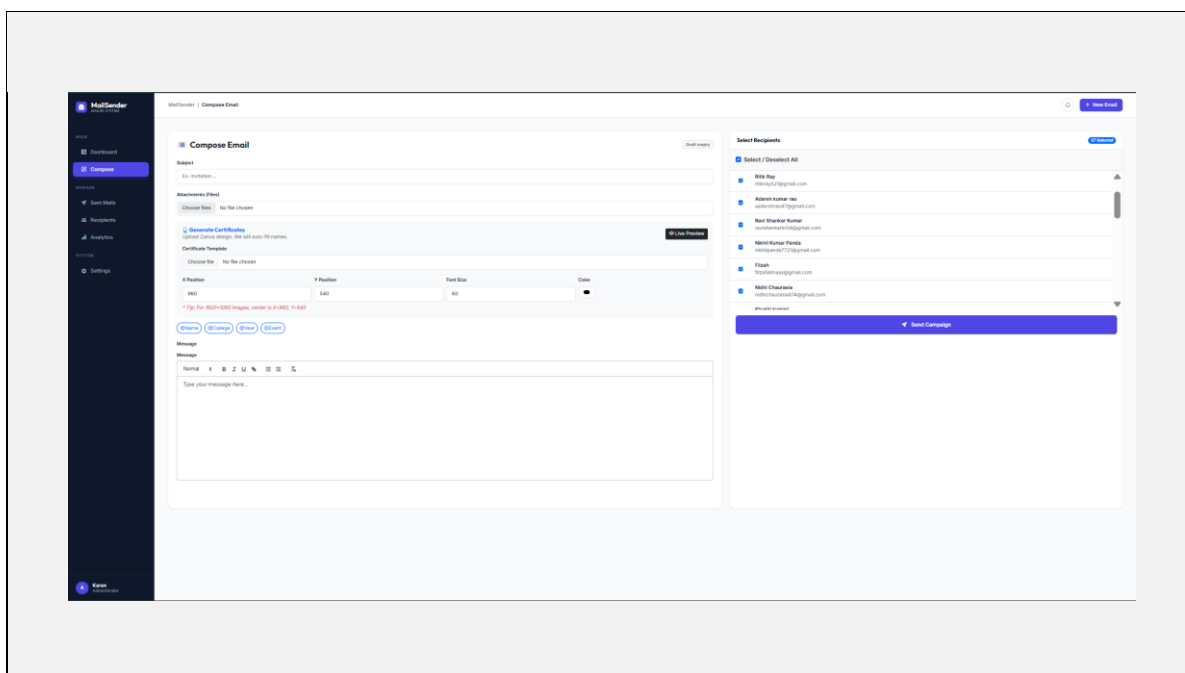
1. Dashboard Overview

The main dashboard displays KPI cards showing total contacts, campaigns sent, emails delivered, and failures. Navigation links to all sections are available in the left sidebar.



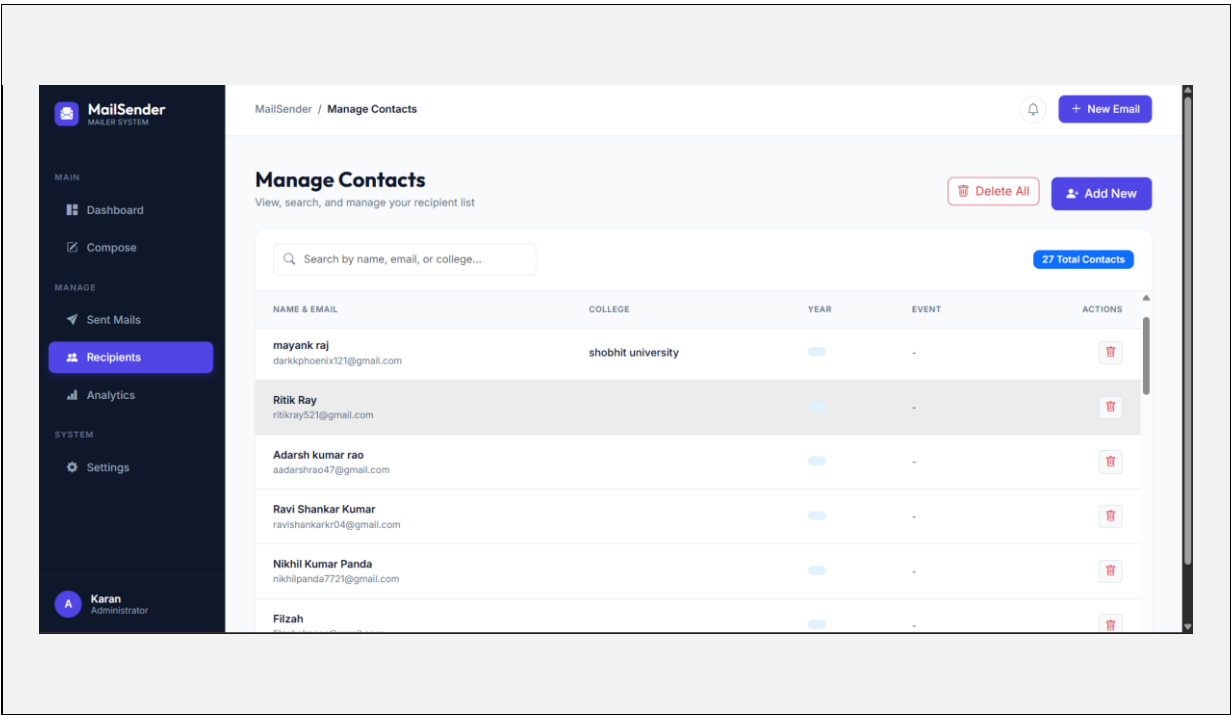
2. Email Composition Page

The Compose page features the Quill.js rich text editor, personalization tag buttons, certificate upload section, and attachment manager.



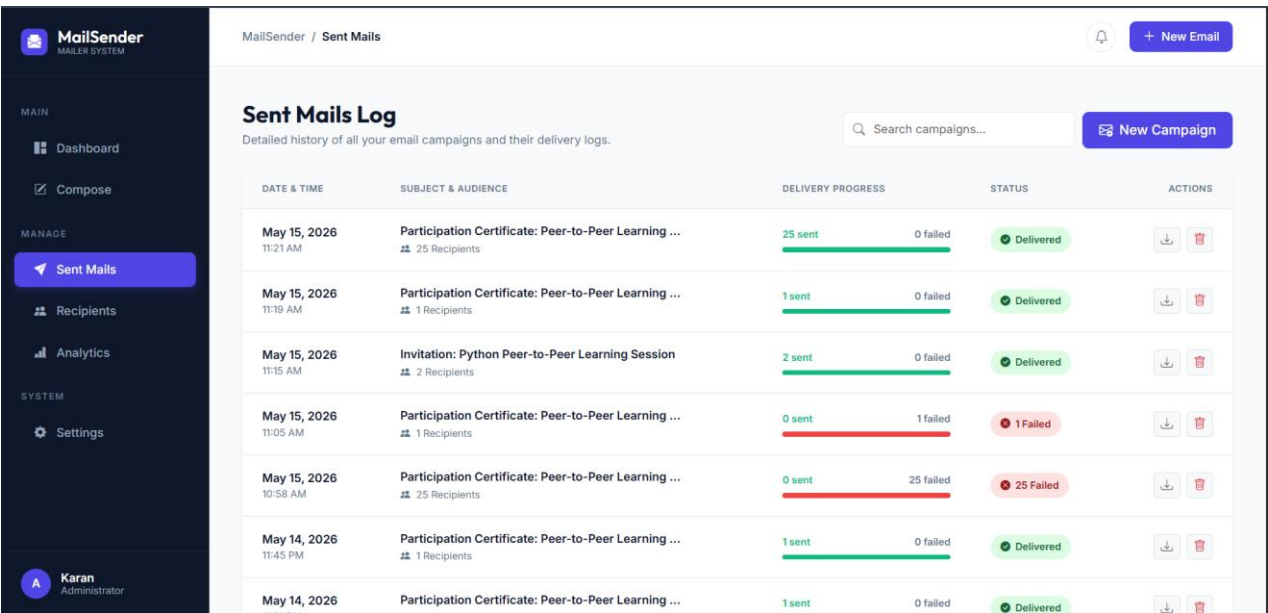
3. Recipient Management

The Contacts page shows a scrollable data grid with all imported recipients. Users can search in real time, add contacts manually, or delete all records.



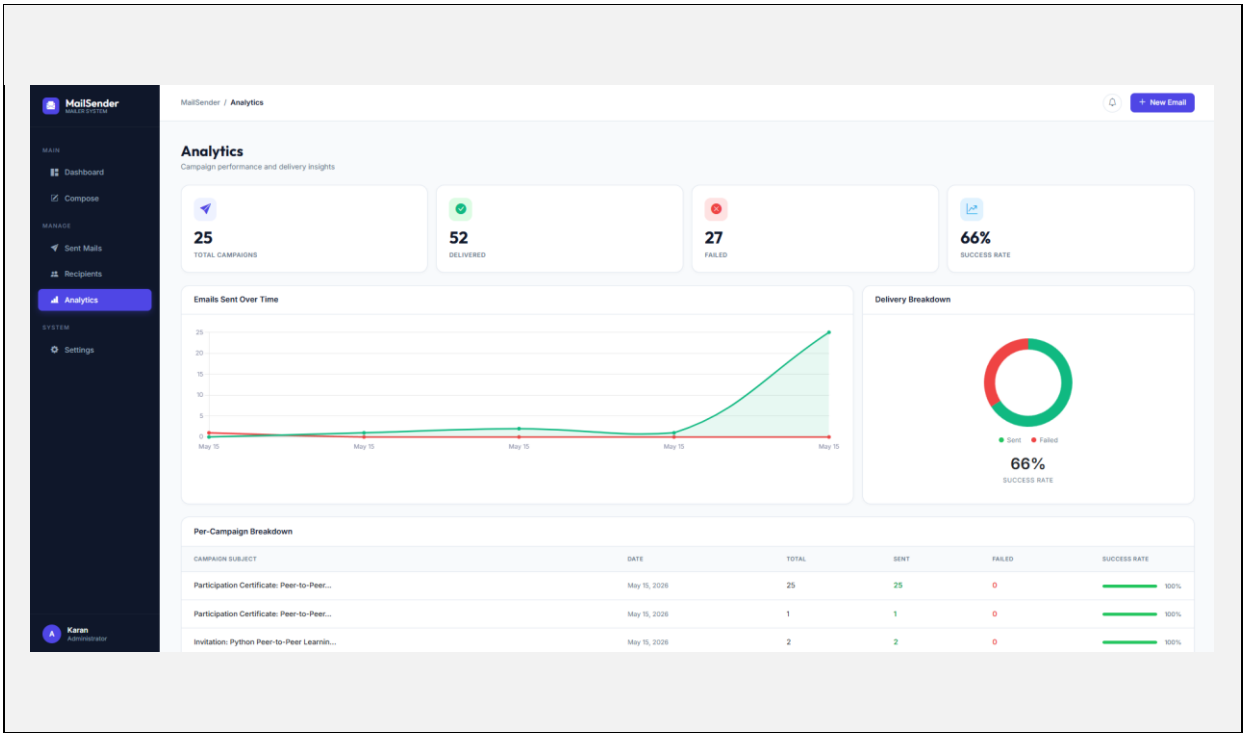
4. Sent Mails Details Page

The Sent Mails page displays a complete delivery log of all email campaigns sent by the system. Users can track delivery status in real time, view successful and failed emails, and monitor campaign performance through detailed records.



5. Analytics Page

The Analytics page displays a Line Chart showing email delivery trends over time and a Donut Chart comparing successful vs. failed deliveries.



6. Settings Page

The Settings page enables users to configure and manage the SMTP email service used by the system. Users can enter SMTP host details, port number, sender email address, TLS preferences, and app passwords, as well as test the connection and view the currently active configuration.

The Settings page allows users to configure their email and system preferences. It includes a sidebar with navigation options: MAIN (Dashboard, Compose), MANAGE (Sent Mails, Recipients, Analytics), and SYSTEM (Settings). The main content area is divided into two sections: 'Email Configuration' and 'Active Configuration'. The 'Email Configuration' section contains fields for SMTP Host (smtp.gmail.com), Port (587), Use TLS (Yes (Recommended)), Sender Email Address (karangupta2114@gmail.com), and App Password (Gmail App Password (16 characters)). The 'Active Configuration' section displays the current settings: SMTP Host (smtp.gmail.com), Port (587), Sender (karangupta2114@gmail.com), and TLS (Enabled). At the bottom, there are buttons for 'Save Settings' and 'Test Connection'.

CONCLUSION

MailForge – SaaS-Based Email Campaign & Analytics Platform successfully demonstrates how modern web technologies can be combined to build a powerful, production-grade bulk email system. The project addresses a real-world problem faced by organizations, educational institutions, and event managers who need to send large volumes of personalized emails efficiently.

By leveraging Django's powerful backend, Python's threading capabilities, in-memory file processing, and a seamless SPA frontend powered by Hotwire Turbo, MailForge achieves both high performance and an excellent user experience. The system can dispatch thousands of personalized emails — complete with dynamically generated PDF certificates — in a fraction of the time required by manual methods.

Key Highlights:

- : High-performance multi-threaded email dispatch using ThreadPoolExecutor
- : Dynamic PDF certificate generation and attachment per recipient
- : Seamless SPA experience with zero page reloads via Hotwire Turbo
- : Real-time campaign analytics with interactive charts
- : Database-driven SMTP configuration without modifying server code
- : Fully in-memory file processing for maximum speed

Future Enhancements:

- **Email Open Tracking:** Implement a 1x1 invisible tracking pixel to monitor recipient open rates
- **Email Templates:** Allow saving frequently used email compositions as reusable templates
- **Unsubscribe Logic:** Automatically append an opt-out link to comply with anti-spam regulations
- **Celery + Redis:** Migrate to a background task queue for enterprise-scale campaigns of 10,000+ emails
- **AI Personalization:** Use AI to suggest optimal email content based on past campaign data

In conclusion, MailForge is a complete, scalable, and professional solution for bulk email campaign management, providing a strong foundation for future enterprise-level enhancements.

REFERENCES

- Django Documentation – <https://docs.djangoproject.com>
- Python Official Documentation – <https://www.python.org/doc/>
- Pillow (PIL) Documentation – <https://pillow.readthedocs.io>
- Quill.js Documentation – <https://quilljs.com/docs/>
- Hotwire Turbo Documentation – <https://turbo.hotwired.dev>
- Chart.js Documentation – <https://www.chartjs.org/docs/>
- Bootstrap 5 Documentation – <https://getbootstrap.com/docs/5.0/>
- PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- Stack Overflow – Community Q&A – <https://stackoverflow.com>
- W3Schools – HTML, CSS & JavaScript Tutorials – <https://www.w3schools.com>

Sincerely,
Gupta Karan & Mayank Raj